MODERN ACADEMY FOR ENGINEERING & TECHNOLOGY

Electronic Eng. and Communication Tech. Branch

DSP- Lab- Mini Project- Fall 27

Adaptive Channel Equalizer

Prepared by:

St. Name	Branch	Section	Level	Law
----	---	---	---	20

Under supervision of:

Dr. / Samir Kamal

Fall 2026 / 2027

Adaptive Channel Equalizer using the LMS Algorithm

1- Project Idea

When digital data are sent through a band-limited channel (telephone line, cable, wireless link), every transmitted symbol is spread in time and overlaps with the neighbouring symbols. This effect is called Inter-Symbol Interference (ISI). Together with the additive noise it makes the receiver take wrong decisions, so the bit error rate (BER) increases.

A channel equalizer is a digital filter placed at the receiver whose job is to undo the distortion of the channel, i.e. its frequency response should be approximately the inverse of the channel response: $H(e^{j\omega}) \cdot W(e^{j\omega}) \approx e^{-j\omega\Delta}$. In practice the channel is unknown and may change with time, so a fixed filter cannot be designed in advance. The solution is an adaptive filter: an FIR filter whose coefficients are adjusted automatically, sample by sample, to minimise the error between its output and the desired (known) training symbols.

In this project an **11-tap adaptive FIR equalizer** is designed and simulated in MATLAB. Its weights are adjusted with the **Least-Mean-Square (LMS) algorithm** of Widrow and Hoff, which is the simplest and most widely used adaptive algorithm. The project studies the convergence of the equalizer (learning curve), the effect of the step size μ, the effect of the amount of channel distortion, the frequency response of the equalizer, and the improvement in the bit error rate.

2- Project Applications

Digital communication receivers: telephone-line and DSL modems, cable modems, Ethernet transceivers.

Wireless and mobile systems: GSM (training sequence in every burst), Wi-Fi and 4G/5G receivers that must track a time-varying multipath channel.

Optical and storage channels: equalization in fibre links and in hard-disk / magnetic recording read channels.

Underwater acoustic communication: channels with long multipath delay spread.

Other uses of the same LMS idea: adaptive echo cancellation in telephones, adaptive noise cancellation (headphones, ECG), and system identification.

3- Project Block Diagram

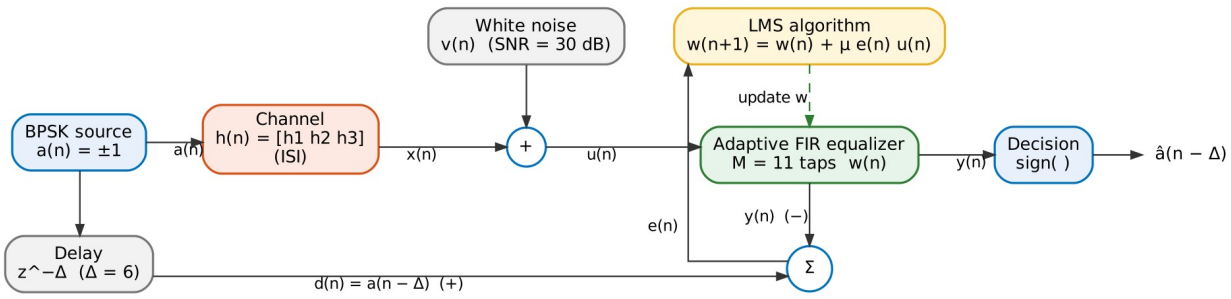


Figure (1): Block diagram of the adaptive channel equalizer (training mode)

Source: random BPSK symbols a(n) = ±1 (training sequence known to the receiver).

Channel: 3-tap FIR filter h(n) that causes ISI; x(n) = h(n) * a(n).

Noise: white Gaussian noise v(n) is added, u(n) = x(n) + v(n), SNR = 30 dB (noise variance = 0.001).

Adaptive equalizer: 11-tap FIR filter, $y(n) = \Sigma\, w_k(n)\, u(n - k),\ k = 0 \ldots 10$.

Desired response: $d(n) = a(n - \Delta)$, the transmitted symbol delayed by $\Delta = 6$ (1 sample channel delay + 5 samples for the centre tap).

Error and LMS block: $e(n) = d(n) - y(n)$ is used by the LMS algorithm to update the weights.

Decision device: $\hat{a} = \text{sign}(y(n))$ gives the detected symbol.

4- Project Steps (Flowchart)

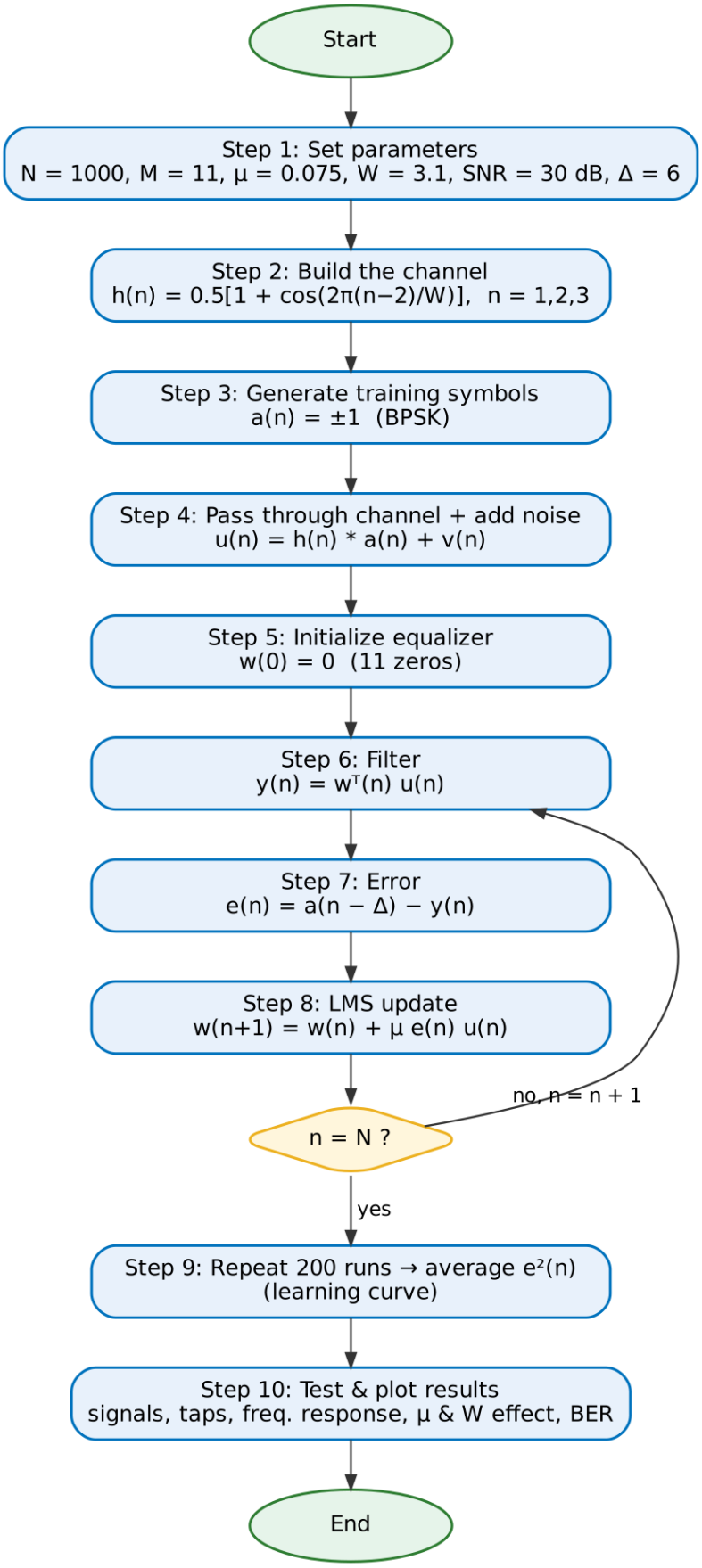


Figure (2): Flowchart of the project

4- 1 Set the parameters

Number of training symbols N = 1000, equalizer length M = 11, step size μ = 0.075, channel parameter W = 3.1, SNR = 30 dB and decision delay Δ = 1 + (M − 1)/2 = 6. The simulation is repeated 200 times to obtain a smooth learning curve.

4- 2 Build the channel

The channel is modelled by a raised-cosine impulse response of three samples:

$$h(n) = \tfrac{1}{2}\,[1 + \cos(2\pi(n-2)/W)]\,, \quad n = 1, 2, 3$$

For W = 3.1: h = [0.2798 1.0 0.2798]. The parameter W controls the amount of distortion: a larger W gives larger side samples h1 = h3, more ISI and a larger eigenvalue spread of the input correlation matrix.

4- 3 Generate the training symbols

a = sign(randn(1, N)) produces a random sequence of +1 and −1 with zero mean and unit variance.

4- 4 Pass the symbols through the channel and add noise

The channel output is the convolution of a(n) with h(n), computed with filter(h, 1, a). Gaussian noise with variance 10^(−SNR/10) = 0.001 is added:

$$u(n) = h1\ a(n) + h2\ a(n-1) + h3\ a(n-2) + v(n)$$

4- 5 Initialize the equalizer

All 11 weights start from zero: w(0) = $[0\ 0\ \ldots\ 0]^T$.

4- 6 Compute the equalizer output

The tap-input vector u(n) = $[u(n)\ u(n-1)\ \ldots\ u(n-10)]^T$ is multiplied by the weights:

$$y(n) = w^T(n)\ u(n)$$

4- 7 Compute the error

The output is compared with the delayed training symbol:

$$e(n) = d(n) - y(n) = a(n-\Delta) - y(n)$$

4- 8 Update the weights (LMS)

The weights move a small step in the direction that reduces the instantaneous squared error (steepest descent with the gradient estimate −2e(n)u(n)):

$$w(n+1) = w(n) + \mu\ e(n)\ u(n)$$

Steps 4-6 to 4-8 are repeated for n = M … N.

4- 9 Average the learning curve

The squared error $e^2(n)$ is averaged over 200 independent runs to estimate the mean square error $J(n) = E[e^2(n)]$.

4- 10 Test and plot the results

The final weights are used to equalize a new received sequence. The program plots: the channel response, the signals before and after equalization, the learning curve, the effect of μ, the effect

of W, the final taps and the overall response h * w, the frequency responses, and the BER with and without the equalizer.

5- Results and Discussion

5- 1 Results

All figures were produced by the MATLAB program in the appendix (N = 1000, M = 11, μ = 0.075, W = 3.1, SNR = 30 dB, 200 runs).

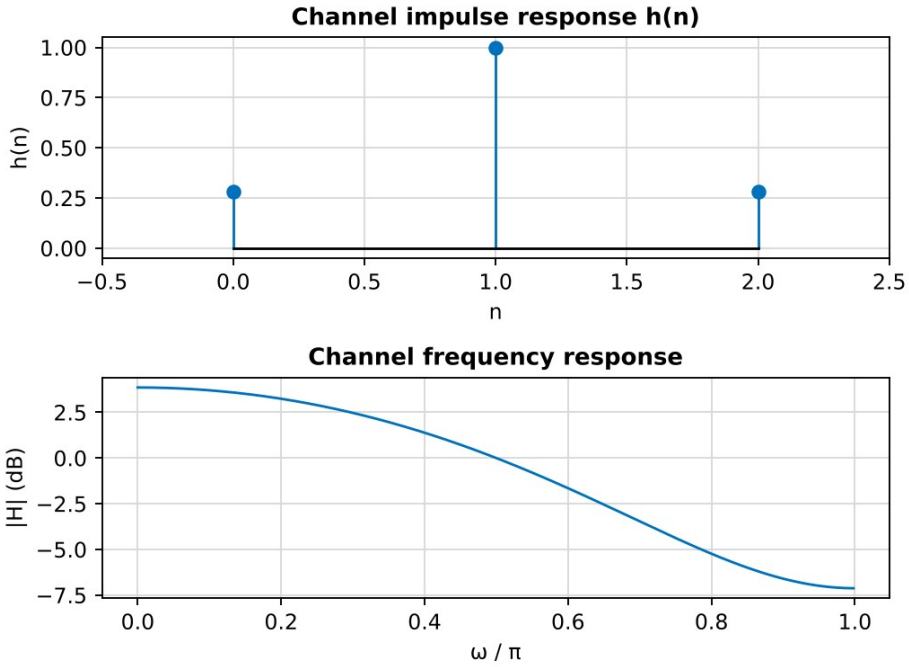


Figure (3): Channel impulse response and frequency response (W = 3.1)

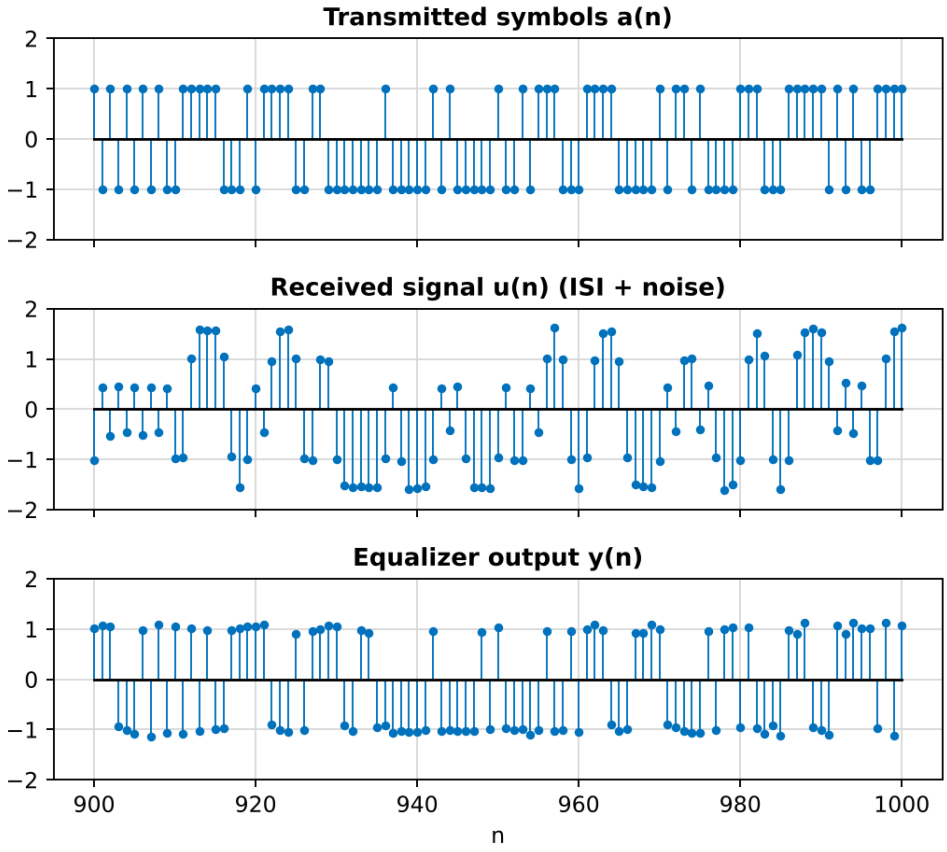


Figure (4): Transmitted symbols, received signal and equalizer output (last 100 samples)

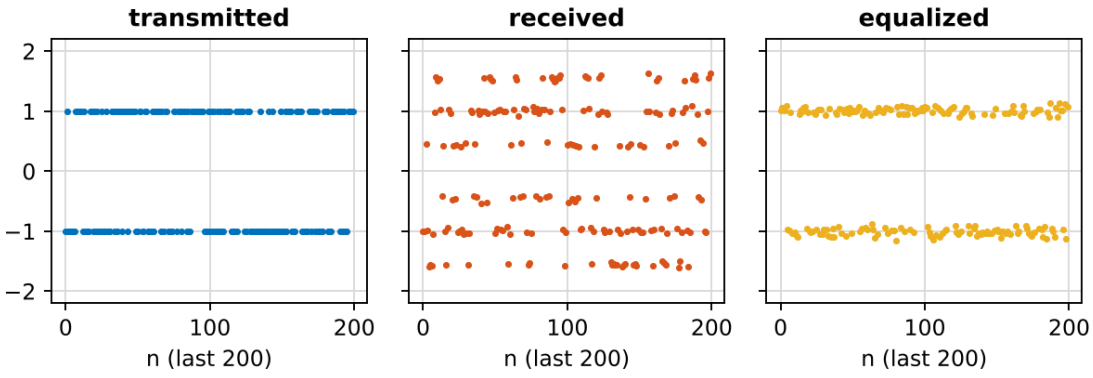


Figure (5): Signal levels: the received signal has 6 levels because of ISI, the equalized signal returns to ±1

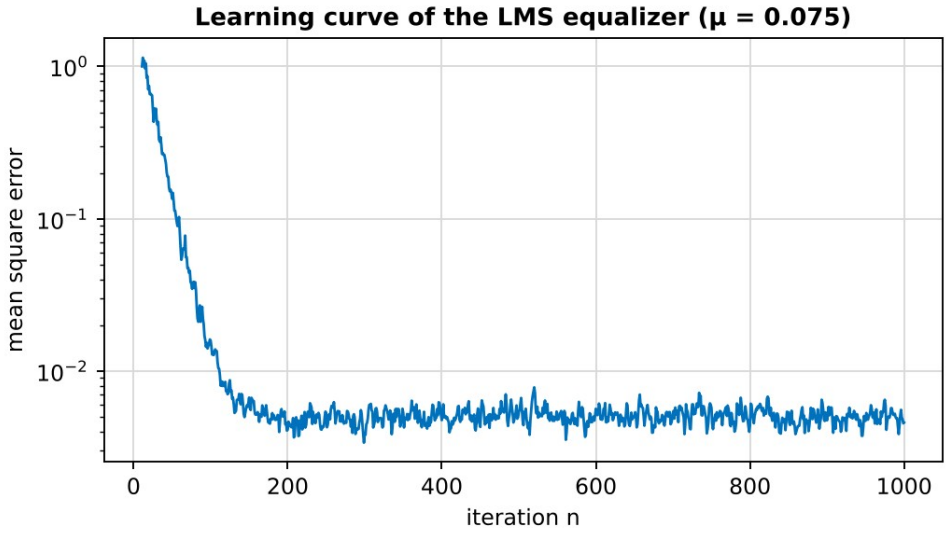


Figure (6): Learning curve of the LMS equalizer (μ = 0.075, average of 200 runs)

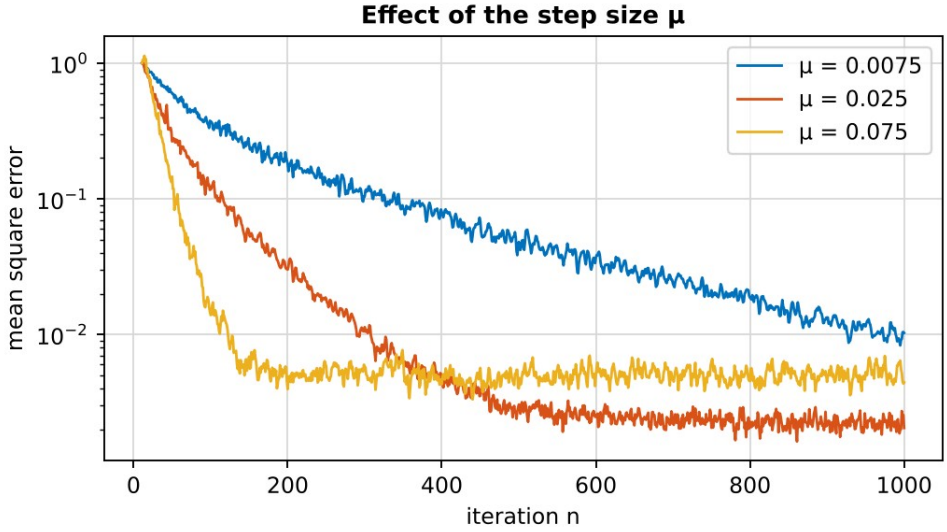


Figure (7): Effect of the step size μ on the learning curve

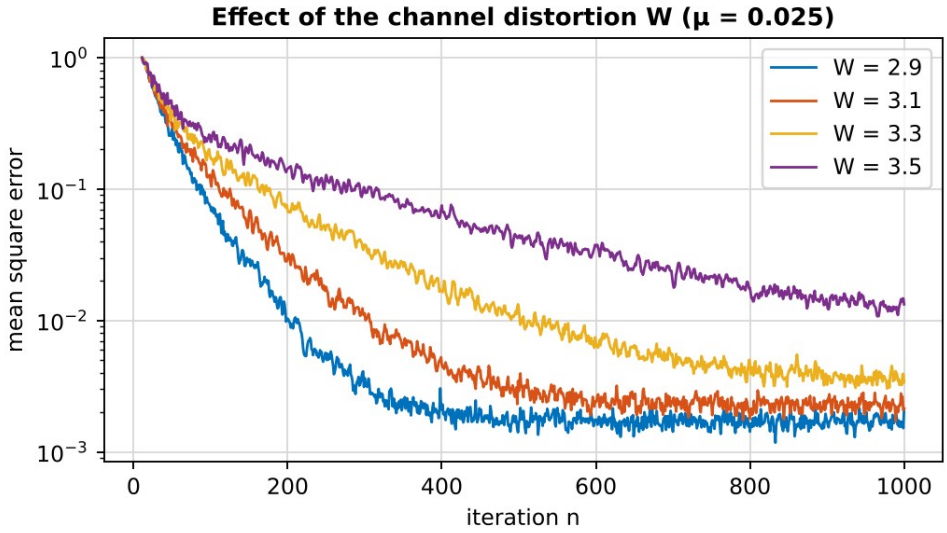


Figure (8): Effect of the channel distortion W (eigenvalue spread) with μ = 0.025

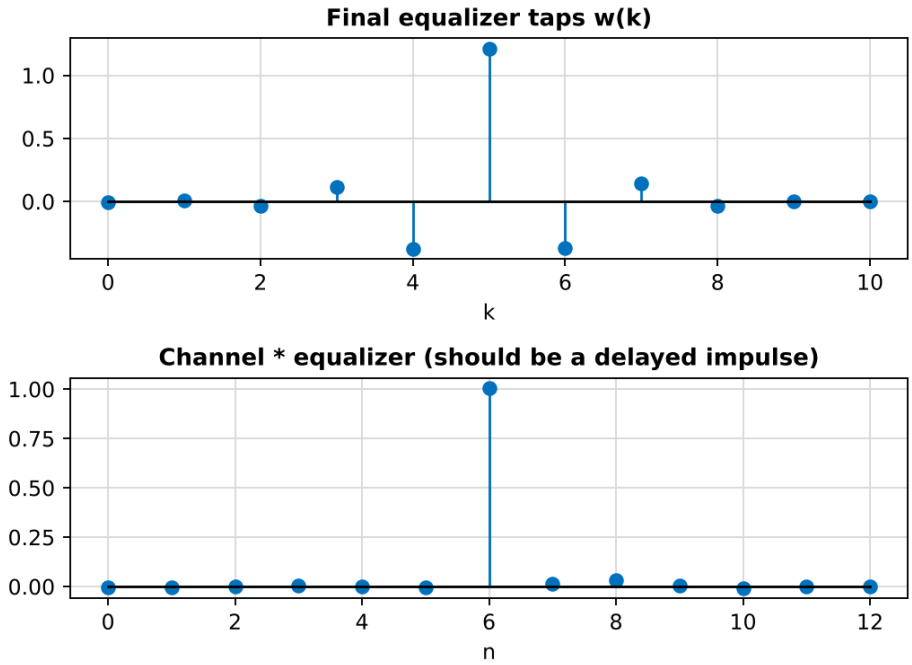


Figure (9): Final equalizer taps and the combined response h(n) * w(n)

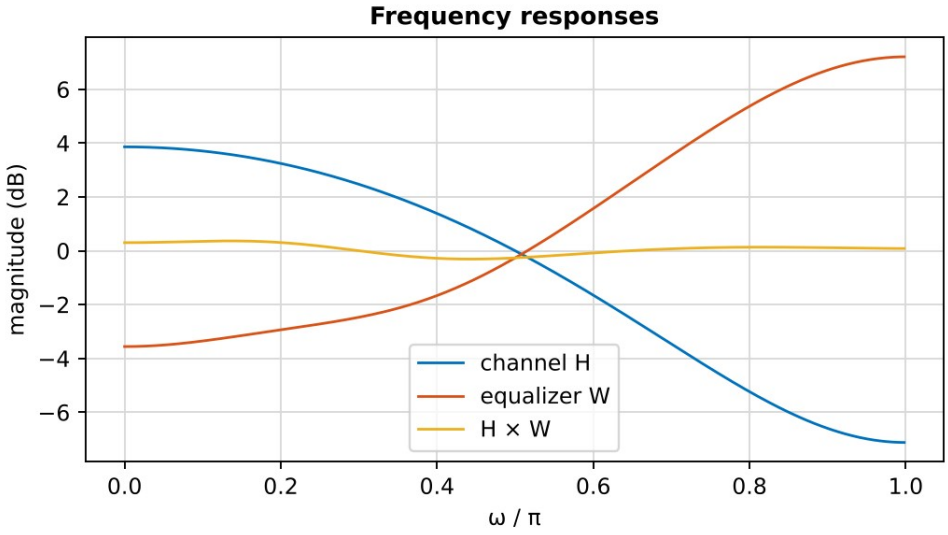


Figure (10): Magnitude responses of the channel, the equalizer and their product

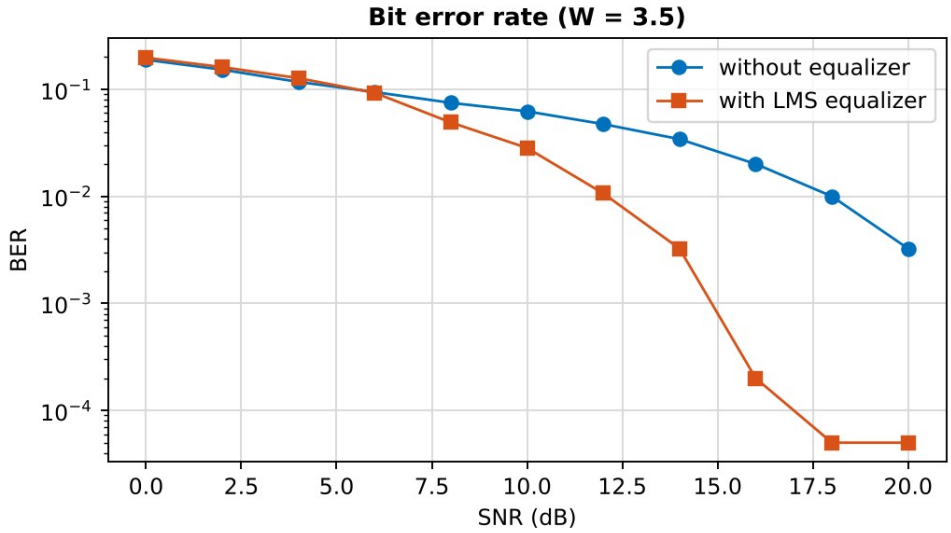


Figure (11): Bit error rate with and without the equalizer (W = 3.5)

Table (1): Main numerical results

Quantity	Value
Channel taps h (W = 3.1)	[0.2798, 1.0, 0.2798]
ISI before equalization (h1 + h3)/h2	0.56
Steady-state MSE (μ = 0.075)	0.0050 (-23.0 dB)
Iterations to converge (μ = 0.075)	≈ 112
Peak of h * w (at n = Δ = 6)	1.006
Residual ISI after equalization (sum of other samples)	0.078
Channel gain range	+3.9 dB … -7.1 dB
Overall gain range |H·W|	-0.31 dB … +0.37 dB

Table (2): Effect of the step size μ (W = 3.1)

μ	Iterations to reach MSE = 0.05	Steady-state MSE
0.0075	≈ 493	0.0131
0.025	≈ 165	0.0022
0.075	≈ 69	0.0051

Table (3): Effect of the channel parameter W (μ = 0.025)

W	h1 = h3	Eigenvalue spread χ(R)	Iterations to reach MSE = 0.05	Steady-state MSE
2.9	0.2194	6.08	≈ 116	0.0017
3.1	0.2798	11.12	≈ 164	0.0023
3.3	0.3365	21.71	≈ 253	0.0038
3.5	0.3887	46.82	≈ 448	0.0146

Table (4): Bit error rate for the channel W = 3.5

SNR (dB)	4	8	10	12	14	16	18
Without equalizer	1.2e-01	7.5e-02	6.2e-02	4.7e-02	3.4e-02	2.0e-02	1.0e-02
With LMS equalizer	1.3e-01	4.9e-02	2.8e-02	1.1e-02	3.3e-03	2.0e-04	5.0e-05

5- 2 Discussion

Effect of ISI: the received signal u(n) is no longer ±1. Because each sample contains h2·a(n−1) plus the two neighbours h1·a(n) and h3·a(n−2), it takes six levels (±1 ± 0.28 ± 0.28), Figure (5). After the equalizer the samples are concentrated again around ±1, which means the ISI has been removed.

Learning curve: the MSE starts at 1 (weights are zero, so e(n) = d(n)) and falls quickly. With μ = 0.075 the equalizer converges after about 112 iterations to a steady-state MSE of about 0.0050 (-23 dB). The curve does not go to zero because of the noise and because the LMS weights keep fluctuating around the optimum Wiener solution (excess MSE or misadjustment).

Effect of μ: a larger step size gives faster convergence (≈ 69 iterations for μ = 0.075 against ≈ 493 for μ = 0.0075) but a higher steady-state error (0.0051 against 0.0022 for μ = 0.025). This is the basic trade-off of the LMS algorithm between speed of adaptation and accuracy (misadjustment). μ must also stay below 2/λmax, otherwise the algorithm diverges.

Effect of channel distortion: increasing W from 2.9 to 3.5 increases the eigenvalue spread of the correlation matrix from 6.1 to 46.8. The LMS algorithm becomes slower (≈ 116 → ≈ 448 iterations) and the final MSE increases, because LMS converges slowly along the directions of the small eigenvalues. A badly distorted channel is therefore harder to equalize.

Equalizer taps: the largest tap is the centre tap w5 ≈ 1.2137, with negative neighbours (≈ -0.3751, -0.3666) that cancel the positive side samples of the channel. The combined response h * w is almost a single impulse of height 1.01 at n = Δ = 6; the remaining ISI is only 0.078 compared with 0.56 before equalization.

Frequency domain: the channel is a low-pass filter (+3.9 dB at ω = 0, -7.1 dB at ω = π). The equalizer learned the opposite shape, a high-pass response (up to +7.2 dB at ω = π), so the product |H·W| is almost flat (within ±0.4 dB). This confirms that the equalizer approximates the inverse of the channel.

Bit error rate: at high SNR the equalizer reduces the BER strongly, e.g. at 14 dB from 3.4e-02 to 3.3e-03 and at 16 dB from 2.0e-02 to 2.0e-04. At very low SNR (0–6 dB) there is no gain: the noise dominates and the equalizer, which boosts high frequencies, also amplifies the noise (noise enhancement).

6- Conclusions

- An 11-tap adaptive FIR equalizer using the LMS algorithm was designed and simulated in MATLAB.
- The equalizer removes the ISI of the channel: the received six-level signal is restored to ±1 and h * w becomes almost an impulse delayed by Δ = 6.
- The LMS algorithm converges in about 112 iterations with μ = 0.075 and reaches an MSE of about -23 dB.
- The step size μ controls the trade-off between convergence speed and steady-state error.
- Larger channel distortion (larger eigenvalue spread) makes LMS converge more slowly and increases the final error.
- In the frequency domain the equalizer approximates the inverse of the channel, so the overall response is flat.

- The equalizer gives a large BER improvement at moderate and high SNR; at very low SNR noise enhancement limits the gain.
- Possible extensions: NLMS or RLS algorithms for faster convergence, decision-directed mode after training, and decision-feedback equalizers.

7- References

[1] S. Haykin, Adaptive Filter Theory, 5th ed., Pearson, 2014 (LMS algorithm and the adaptive equalization computer experiment).

[2] B. Widrow and S. D. Stearns, Adaptive Signal Processing, Prentice Hall, 1985.

[3] B. Widrow and M. E. Hoff, "Adaptive switching circuits," IRE WESCON Convention Record, Part 4, pp. 96–104, 1960.

[4] S. U. H. Qureshi, "Adaptive equalization," Proceedings of the IEEE, vol. 73, no. 9, pp. 1349–1387, Sept. 1985.

[5] J. G. Proakis and M. Salehi, Digital Communications, 5th ed., McGraw-Hill, 2008.

[6] A. H. Sayed, Fundamentals of Adaptive Filtering, Wiley-IEEE Press, 2003.

[7] J. G. Proakis and D. G. Manolakis, Digital Signal Processing: Principles, Algorithms, and Applications, 4th ed., Pearson, 2007.

[8] MathWorks, MATLAB and Signal Processing Toolbox documentation (filter, freqz, conv), www.mathworks.com.

Appendix: MATLAB Code

```
%% DSP Mini Project : Adaptive Channel Equalizer using the LMS algorithm
% A random BPSK sequence (+1/-1) is sent through a channel that causes
% inter-symbol interference (ISI) plus white Gaussian noise.
% An 11-tap FIR filter adapts its weights with the LMS algorithm so that
% its output becomes equal to the transmitted (delayed) symbol.
clc; clear; close all;

%% 1) Parameters
N     = 1000;          % number of training symbols (iterations)
M     = 11;            % number of equalizer taps
mu    = 0.075;         % LMS step size
W     = 3.1;           % channel parameter (bigger W = more distortion)
SNRdB = 30;            % signal to noise ratio in dB
runs  = 200;           % independent runs to average the learning curve
delay = 1 + (M-1)/2;   % total delay = channel delay (1) + equalizer centre (5) = 6

%% 2) Channel impulse response (raised cosine)
h = channel(W);                          % h = [h1 h2 h3]
disp('Channel impulse response h(n) = '); disp(h)

%% 3) Learning curve (MSE averaged over many runs)
MSE = zeros(1,N);
for r = 1:runs
    [e, w] = lms_equalizer(h, N, M, mu, delay, SNRdB);
    MSE = MSE + e.^2;
end
MSE = MSE / runs;

%% 4) One run to look at the signals before and after the equalizer
a     = sign(randn(1,N));                          % transmitted symbols
x     = filter(h, 1, a);                           % channel output (with ISI)
u     = x + sqrt(10^(-SNRdB/10)) * randn(1,N);     % received signal (with noise)
yeq   = filter(w, 1, u);                           % equalizer output (final taps)

%% 5) Figures
% Figure 1 : channel
figure(1)
subplot(2,1,1); stem(0:2, h, 'filled'); grid on
title('Channel impulse response h(n)'); xlabel('n'); ylabel('h(n)')
[Hc, f] = freqz(h, 1, 512);
subplot(2,1,2); plot(f/pi, 20*log10(abs(Hc))); grid on
title('Channel frequency response'); xlabel('\omega / \pi'); ylabel('|H| (dB)')

% Figure 2 : signals before and after equalization
figure(2)
k = N-100:N;
subplot(3,1,1); stem(k, a(k), '.'); title('Transmitted symbols a(n)'); ylim([-2 2])
subplot(3,1,2); stem(k, u(k), '.'); title('Received signal u(n) (ISI + noise)'); ylim([-2
2])
subplot(3,1,3); stem(k, yeq(k), '.'); title('Equalizer output y(n)'); ylim([-2 2]);
xlabel('n')

% Figure 3 : learning curve
figure(3)
semilogy(M:N, MSE(M:N)); grid on
title(['Learning curve of the LMS equalizer (\mu = ' num2str(mu) ')'])
xlabel('iteration n'); ylabel('mean square error')

% Figure 4 : effect of the step size mu
figure(4)
mus = [0.0075 0.025 0.075];
for i = 1:length(mus)
    MSEi = zeros(1,N);
    for r = 1:runs
        e = lms_equalizer(h, N, M, mus(i), delay, SNRdB);
        MSEi = MSEi + e.^2;
    end
    semilogy(M:N, MSEi(M:N)/runs); hold on
```

```
end
grid on; legend('\mu = 0.0075', '\mu = 0.025', '\mu = 0.075')
title('Effect of the step size \mu'); xlabel('iteration n'); ylabel('mean square error')

% Figure 5 : effect of the channel distortion W (eigenvalue spread)
figure(5)
Ws = [2.9 3.1 3.3 3.5];
for i = 1:length(Ws)
    hi = channel(Ws(i));
    MSEi = zeros(1,N);
    for r = 1:runs
        e = lms_equalizer(hi, N, M, 0.025, delay, SNRdB);
        MSEi = MSEi + e.^2;
    end
    semilogy(M:N, MSEi(M:N)/runs); hold on
    % eigenvalue spread of the input correlation matrix
    r0 = sum(hi.^2) + 10^(-SNRdB/10);
    r1 = hi(1)*hi(2) + hi(2)*hi(3);
    r2 = hi(1)*hi(3);
    R  = toeplitz([r0 r1 r2 zeros(1,M-3)]);
    fprintf('W = %.1f   eigenvalue spread = %.2f\n', Ws(i), max(eig(R))/min(eig(R)));
end
grid on; legend('W = 2.9', 'W = 3.1', 'W = 3.3', 'W = 3.5')
title('Effect of the channel distortion W (\mu = 0.025)'); xlabel('iteration n');
ylabel('mean square error')

% Figure 6 : equalizer taps and channel + equalizer response
figure(6)
subplot(2,1,1); stem(0:M-1, w, 'filled'); grid on
title('Final equalizer taps w(k)'); xlabel('k')
subplot(2,1,2); stem(0:M+1, conv(h, w), 'filled'); grid on
title('Channel * equalizer (should be a delayed impulse)'); xlabel('n')

% Figure 7 : frequency responses
figure(7)
[He, f] = freqz(w, 1, 512);
plot(f/pi, 20*log10(abs(Hc)), f/pi, 20*log10(abs(He)), f/pi, 20*log10(abs(Hc.*He))); grid
on
legend('channel H', 'equalizer W', 'H \times W')
title('Frequency responses'); xlabel('\omega / \pi'); ylabel('magnitude (dB)')

% Figure 8 : bit error rate with and without the equalizer
figure(8)
SNRs = 0:2:20;  Nb = 20000;  hb = channel(3.5);
BER0 = zeros(size(SNRs));  BER1 = zeros(size(SNRs));
for i = 1:length(SNRs)
    [~, wb] = lms_equalizer(hb, 2000, M, 0.01, delay, SNRs(i));   % train
    a  = sign(randn(1,Nb));
    u  = filter(hb, 1, a) + sqrt(10^(-SNRs(i)/10)) * randn(1,Nb);
    d0 = sign(u(M+1:end));                          % no equalizer (channel delay = 1)
    BER0(i) = mean(d0 ~= a(M:end-1));
    y  = filter(wb, 1, u);
    d1 = sign(y(M+1:end));                          % with equalizer (delay = 6)
    BER1(i) = mean(d1 ~= a(M+1-delay:end-delay));
end
semilogy(SNRs, BER0, 'o-', SNRs, BER1, 's-'); grid on
legend('without equalizer', 'with LMS equalizer')
title('Bit error rate (W = 3.5)'); xlabel('SNR (dB)'); ylabel('BER')

%% ---------------- local functions ----------------
function h = channel(W)
% raised cosine channel  h(n) = 0.5*(1 + cos(2*pi/W*(n-2))) , n = 1,2,3
n = 1:3;
h = 0.5 * (1 + cos(2*pi/W*(n-2)));
end

function [e, w] = lms_equalizer(h, N, M, mu, delay, SNRdB)
% trains an M-tap equalizer with LMS and returns the error e(n) and final taps w
a = sign(randn(1,N));                                % training symbols
u = filter(h, 1, a) + sqrt(10^(-SNRdB/10)) * randn(1,N);
```

```
w = zeros(M,1);                                     % start from zero weights
e = zeros(1,N);
for n = M:N
    un   = u(n:-1:n-M+1).';      % tap-input vector [u(n) ... u(n-M+1)]
    y    = w.' * un;             % equalizer output
    d    = a(n-delay);           % desired response = delayed symbol
    e(n) = d - y;                % error
    w    = w + mu * e(n) * un;   % LMS weight update
end
end
```